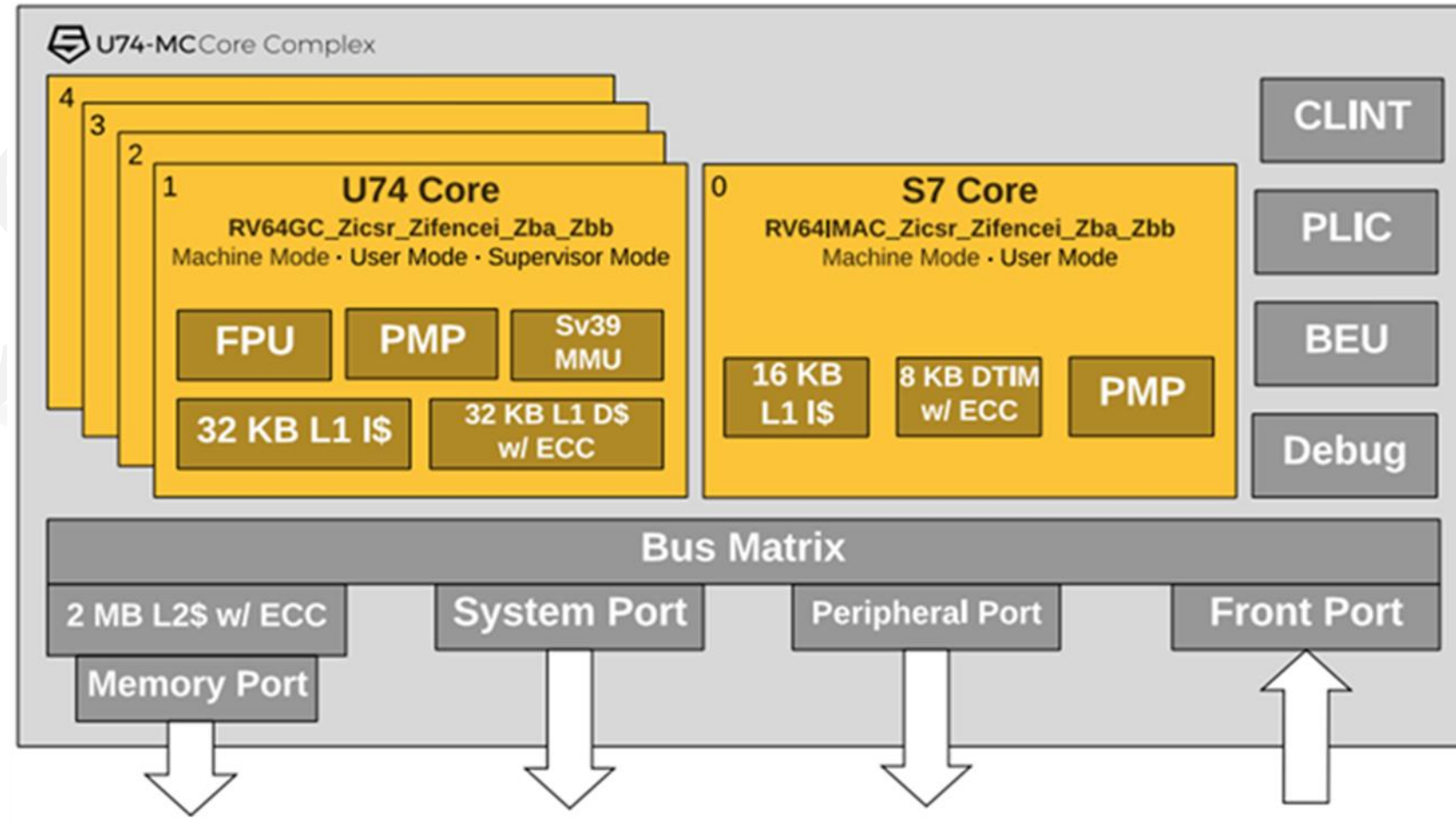


CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 10

Memory System



U74 Memory System

- Instruction Memory System

- L1 Instruction Cache

- The L1 instruction cache is a 32 KiB 4-way set associative cache.
 - It is **virtually-indexed, physically-tagged** with a line size of 64 bytes and is **read/write-allocate** with a random replacement policy. a random line is chosen to be evicted
 - There are **no write-backs** to memory from the instruction cache and it is **not kept coherent** with the memory system or with instruction caches of other processors
 - The L1 instruction cache has **parity error protection** support.
 - **As instruction fetch is extremely timing-sensitive** the critical path doesn't allow for ECC so there is only **parity check** which can detect odd number of bit errors
 - **Once error is detected**, we can invalidate the cache line and **refetch from L2 cache** which is **ECC protected**

U74 Memory System

- Data Memory System

- L1 Data Cache

- The L1 data cache is a 32 KiB 8-way set associative cache.
 - It is virtually-indexed, physically-tagged with a line size of 64 bytes and is read/write-allocate with a random replacement policy.
 - The cache operates in write-back mode; this means that if a cache line is dirty, it is written back to memory when evicted.
 - A cache line fill triggers a burst access starting with the first address of the cache line.
 - The ECC hardware includes logic for detection and correction, in addition to 7 redundant bits per 32-bit codeword for both Data and Tag for every 32 bit of data 7 bits of ECC are stored summing up to 39 bits codeword
 - When a block of data is allocated to the L1 cache, it is also allocated to the L2 cache. When a block is evicted from the L1, the corresponding line in the L2 is then updated and marked dirty.



when L1 data cache misses, instead of fetching one byte to L1 data cache whole cache line 64B is added this is called memory burst

L1 updated, L1 line evicted, updated the line in L2, later L2 will update the line in MM

Memory System RISC-V Core

- Tightly-Integrated Memory (Tightly Coupled Memory)
 - A small, deterministic, low-latency SRAM block that is directly connected to the CPU and used only for fast access to time-critical code, often in real-time systems.
 - Data Tightly-Integrated Memory (DTIM)
 - It is memory mapped i.e. a fixed address space is allocated for these memories
 - Data must be explicitly placed in DTIM through software

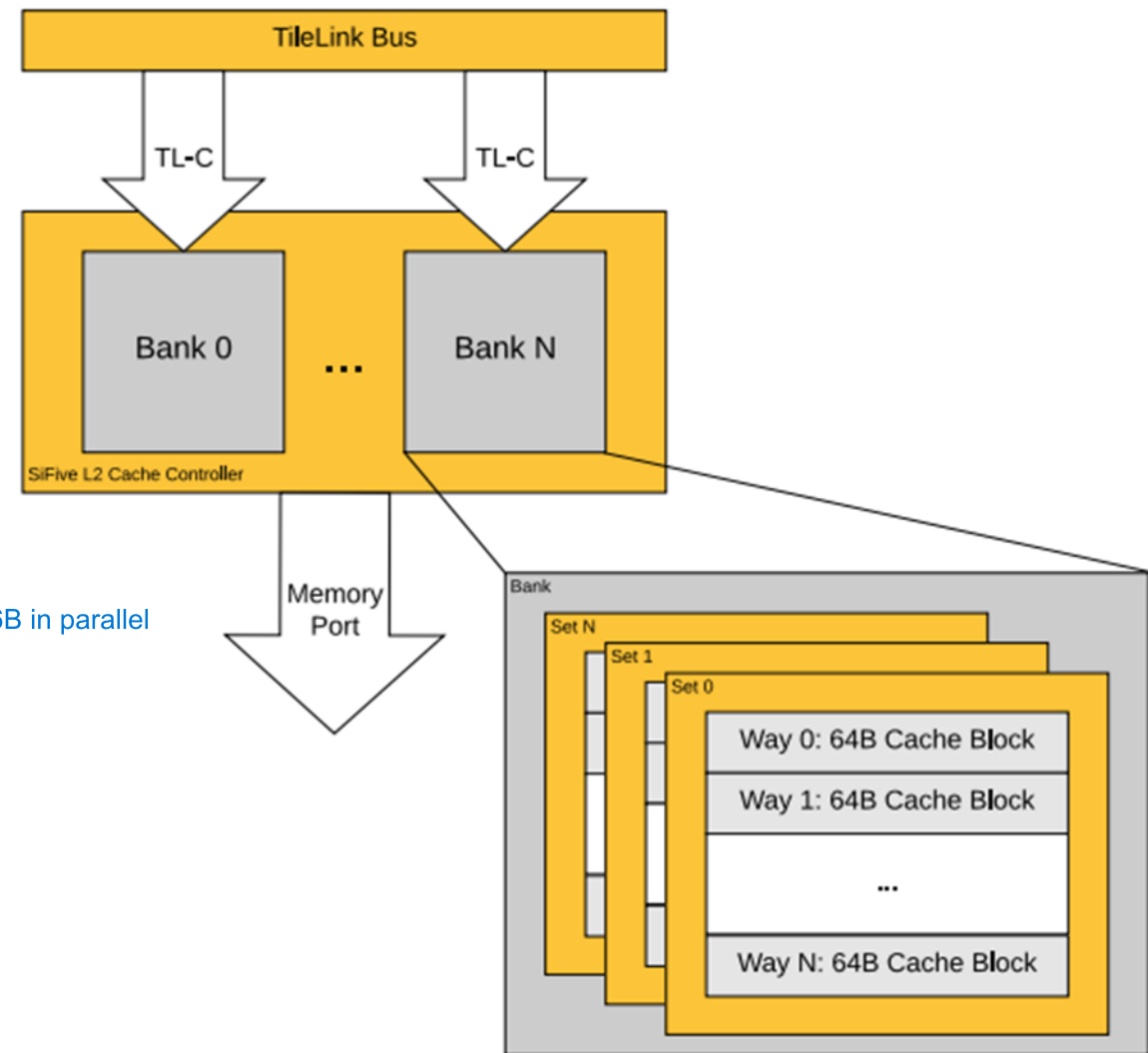
is Tightly Integrated Memory = on-chip or near-core memory with very low latency access compared to external DRAM
Registers → L1 Cache → L2 Cache → Tightly Integrated Memory → DRAM → Storage
SRam is used for caches, it stores data using flipflops, hence no refresh needed

U74 Memory System

- Data Memory System

- L2 Cache

- L2 Cache Controller is configured into 4 banks. Each bank contains 512 sets of 16 ways and each way contains a 64-byte block.
 - $4 \times 512 \times 16 \times 64 = 2 \text{ MiB}$
 - Each bank has its own dedicated 128-bit TL-C inner port. As such, multiple requests by CPUs to different banks may proceed in parallel. it can read 128 bits = 16B in parallel
 - The outer port of the L2 Cache Controller is a 256-bit TL-C port shared among all banks and typically connected to a DDR controller (cacheable memory).



U74 Memory System

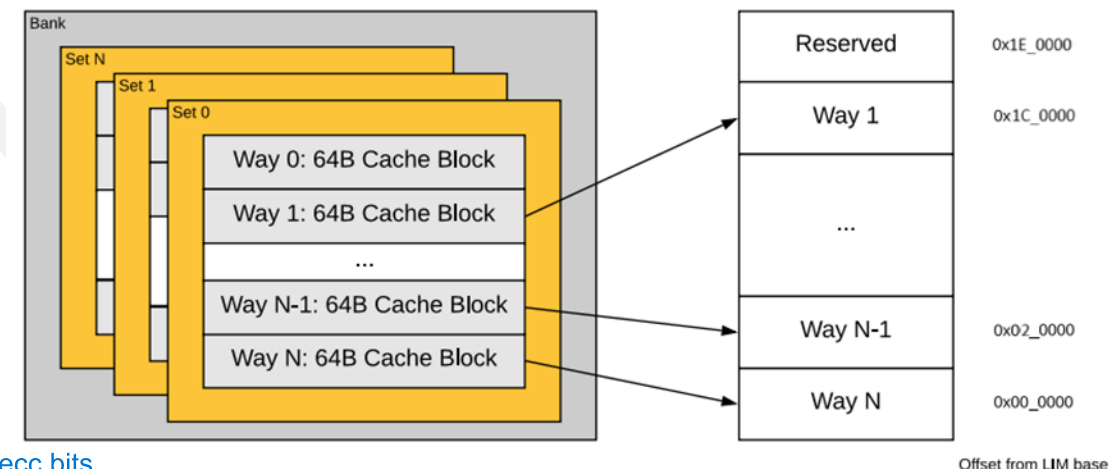
• Data Memory System

Loosely Integrated Memory refers to memory that is not tightly coupled to the CPU core, meaning it is accessed through external interfaces (like a memory controller) and is typically off-chip and higher latency.

– (L2 – LIM) Loosely Integrated Memory

- Level 2 Cache Controller allows for its SRAMs to act either as direct addressed memory or as a cache
- When cache ways are disabled, they are addressable in the L2 Loosely Integrated Memory (L2-LIM) address space. Fetching instructions or data from the L2-LIM provides deterministic behavior equivalent to an L2 cache hit, with no possibility of a cache miss.
- After reset, all ways, except for way 0, are disabled. Cache ways can be enabled by writing to the WayEnable register.
- The highest numbered L2 Cache Way is mapped to the lowest L2-LIM address space, and way 1 occupies the highest L2-LIM address range. As L2 cache ways are enabled, the size of the L2-LIM address space shrinks.
- The ECC hardware includes logic for detection and correction, in addition to 8 redundant bits per 64-bit codeword for both Data and Tag

for every 64 bit word we have 8 ecc bits



Offset from LIM base

CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 11

Memory System RISC-V Core

- L2 Zero Device (L2 Scratchpad)
 - L2 Zero device is a memory-mapped region that **behaves like read/write memory**, and always returns zeros when read and discards all writes
 - Writes to the scratchpad region allocate into cache, however, as there is no memory backing this address space, cache evictions from addresses in the scratchpad result in data loss.
 - The main advantage of the L2 Scratchpad over the L2-LIM is that it is a cacheable region allowing for data stored to the scratchpad to also be cached in a master's L1 data cache resulting in faster access.

If you read from it → you always get 0x00000000
If you write to it → the data is silently thrown away
It's essentially a "dummy" memory region, useful for initializing or testing

When you write to the scratchpad address range, the data goes into the L2 cache
However, since there's no actual backing memory (no RAM behind it), if the cache needs to evict that data → it's gone forever (data loss)
So you must be careful — don't rely on data surviving a cache eviction

Unlike the L2-LIM (Loosely Integrated Memory), the scratchpad is cacheable
This means data can also be stored in the master's L1 data cache
L1 is closer to the CPU core → faster access speeds

ECC – Error Correcting Code

- Considering n bits the total valid codewords are 2^n . The Hamming distance for this is 1.
- A single bit change, however, results in another codeword. What if we have codewords which become invalid if a single bit change happens. This way we can know whether a bit has changed or not.

– E.g. for 3-bits valid codes are 000 to 111, if we add one more bit the codes will be 0000 to 1111 but we keep only half of them as valid codewords and half as invalid.

– Valid: 0000, 0011, 0101, 0110, 1001, 1010, 1100, 1111
 0 3 5 6 1 2 4 7

How many r bits? # cases?
 1 – no error
 1 – error somewhere
 $2^r \geq 1+1$, so $r = 1$

- So now if a single bit change occurs we can easily detect it.
- This extra bit that we have added is called the Parity bit. Hamming distance for these codewords is 2.

$$d_{\min} \geq k + 1 \text{ to detect } k \text{ errors}$$

- What if we want to correct k errors, then the code needs to know the shortest distance between two valid codes so

$$d_{\min} \geq 2k + 1 \text{ to correct } k \text{ errors}$$

ECC – Error Correcting Code

- SECDED (Single Error Correction, Double Error Detection)

- For a data word D, we compute parity bits P such that: $2^r \geq n + r + 1$

Where

- n = data bits
- r = parity bits (redundant bits)

- Each parity bit covers a subset of positions

Example

- $P_1 \rightarrow$ checks bits whose index has $LSB = 1$
- $P_2 \rightarrow$ checks bits whose 2nd bit = 1

- How many bits do we need for 32 data bits?

- $r = 5: 32 + 5 + 1 = 38 \leq 32 \times$
- $r = 6: 32 + 6 + 1 = 39 \leq 64 \checkmark$ since for k bit we need $k+1 \leq d_{min}$
- Add 1 extra overall parity bit $\rightarrow$ detect double-bit errors
- 32-bit data $\rightarrow$ 7 ECC bits (total 39 bits)

How many cases are there?

1 - No error

n - Error in any one of the n bits

r - Error in any one of the r bits

$$2^r \geq n + r + 1$$

case = each possibility of bit got corrupted is a case
this formula represents that there should be enough cases to represent all cases

$P_i = \text{XOR of selected data bits}$

ECC – Error Correcting Code

- Example using 8-bit data
 1. Compute all parity bits
 2. Compute global parity
 3. Inject a single-bit error
 4. Compute syndrome → correct the error

Data = 10110011

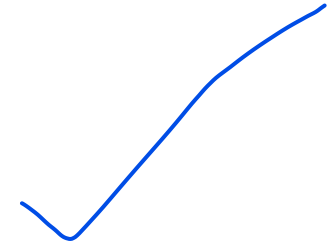
Number of Parity bits : $n + r + 1 \leq 2^r$

$r = 4$ plus 1 global parity

Pg P8 P4 P2 P1

Encode data

13	12	11	10	9	8	7	6	5	4	3	2	1
Pg	D7	D6	D5	D4	P8	D3	D2	D1	P4	D0	P2	P1
	1	0	1	1		0	0	1		1		



ECC – Error Correcting Code

- Example using 8-bit data
 1. Compute all parity bits
 2. Compute global parity
 3. Inject a single-bit error
 4. Compute syndrome → correct the error

13	12	11	10	9	8	7	6	5	4	3	2	1
Pg	D7	D6	D5	D4	P8	D3	D2	D1	P4	D0	P2	P1
	1	0	1	1		0	0	1		1		

P1 (positions 1,3,5,7,9,11)

$$P1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0$$

$$P1 = 1$$

odd number of ones = 1

P2 (positions 2,3,6,7,10,11)

$$P2 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0$$

$$P2 = 0$$

P4 (positions 4,5,6,7,12)

$$P4 \oplus 1 \oplus 0 \oplus 0 \oplus 1$$

$$P4 = 0$$

P8 (positions 8,9,10,11,12)

$$P8 \oplus 1 \oplus 1 \oplus 0 \oplus 1$$

$$P8 = 1$$

ECC – Error Correcting Code

- Example using 8-bit data
 1. Compute all parity bits
 2. Compute global parity
 3. Inject a single-bit error
 4. Compute syndrome → correct the error

13	12	11	10	9	8	7	6	5	4	3	2	1
Pg	D7	D6	D5	D4	P8	D3	D2	D1	P4	D0	P2	P1
	1	0	1	1	1	0	0	1	0	1	0	1

Pg: XOR all 12 bits

Odd 1s, so Pg = 1

Pg	P8	P4	P2	P1	D7	D6	D5	D4	D3	D2	D1	D0
1	1	0	0	1	1	0	1	1	0	0	1	1

ECC – Error Correcting Code

- Example using 8-bit data
 1. Compute all parity bits
 2. Compute global parity
 3. Inject a single-bit error
 4. Compute syndrome → correct the error

13	12	11	10	9	8	7	6	5	4	3	2	1
Pg	D7	D6	D5	D4	P8	D3	D2	D1	P4	D0	P2	P1
1	1	0	1	1	1	0	1	1	0	1	0	1

Flip 6th bit position from 0 to 1

ECC – Error Correcting Code

- Example using 8-bit data

1. Compute all parity bits
2. Compute global parity
3. Inject a single-bit error
4. Compute syndrome → correct the error

13	12	11	10	9	8	7	6	5	4	3	2	1
Pg	D7	D6	D5	D4	P8	D3	D2	D1	P4	D0	P2	P1
1	1	0	1	1	1	0	0	1	0	1	0	1

S1 (positions 1,3,5,7,9,11)

$$1 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0$$

$$S1 = 0$$

S2 (positions 2,3,6,7,10,11)

$$0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0$$

$$S2 = 1$$

S4 (positions 4,5,6,7,12)

$$0 \oplus 1 \oplus 1 \oplus 0 \oplus 1$$

$$S4 = 1$$

S8 (positions 8,9,10,11,12)

$$1 \oplus 1 \oplus 1 \oplus 0 \oplus 1$$

$$S8 = 0$$

Final Syndrome
 $S = S8 \ S4 \ S2 \ S1$
 $= 0 \ 1 \ 1 \ 0$

6th bit is wrong